

EdgeBlock: A Fixed-Size Block Graph Format for Incremental Algorithms on Unified Memory

Authors: Lu Yan (闫路) Kaizhuo Chen (陈楷卓)

Date: July 7, 2026

Repository: github.com/LittleLollipop/axolotl

Abstract

We present EdgeBlock, a graph data structure designed from the ground up for unified memory architectures. EdgeBlock organizes adjacency lists into fixed-size blocks of 32 edges, matching the GPU warp width to achieve coalesced memory access without data transfer overhead between CPU and GPU. We implement five incremental graph algorithms—PageRank, BFS, SSSP, Connected Components, and Triangle Counting—on top of EdgeBlock, following a CPU-GPU collaborative pattern: the CPU identifies affected vertices, the GPU executes parallel computation on those vertices, and the CPU checks for convergence. On an Apple M4 with 10 GPU cores, incremental BFS achieves a 1580× speedup over full recomputation (50K vertices, 250K edges), and incremental Connected Components achieves 4920×. The EdgeBlock format reduces data loading time by 61–83% compared to CSR-based pipelines by eliminating intermediate format conversion. We release the full implementation as open-source Rust library with Python bindings.

1. Introduction

Graph processing is at the core of modern applications—from social network analysis and recommendation systems to knowledge graphs and fraud detection.

As graphs grow in size, two challenges dominate: (1) how to store graph data efficiently for both CPU and GPU access, and (2) how to update algorithm results incrementally when the graph changes, rather than recomputing from scratch.

Traditional graph processing frameworks address these challenges separately. GPU-accelerated libraries like Gunrock [1] and cuGraph use CSR (Compressed Sparse Row) format, which requires explicit data copies between host and device memory. Incremental algorithms such as those in GraphBolt [2] and KickStarter [3] provide update mechanisms but were designed for discrete GPU architectures (CUDA) with explicit memory management.

Unified memory architectures—where CPU and GPU share a single physical memory pool—fundamentally change this landscape. On unified memory chips (Apple M-series, Intel Lunar Lake, NVIDIA Grace), data structures can be accessed by both CPU and GPU without copying. However, CSR format, while compact, suffers from non-coalesced GPU memory access patterns: adjacent threads in a warp access non-adjacent memory locations, wasting the GPU's memory bandwidth.

A deeper consequence of unified memory is that the implicit assumptions underpinning decades of systems design begin to unravel. CPU and GPU represent two radically different computational models—one optimized for low-latency sequential execution with abundant per-thread memory, the other for high-throughput parallel execution with minimal per-thread state—and these models impose opposing constraints on data structures. Classic designs were forced to choose: CSR is GPU-friendly but hostile to CPU updates; adjacency lists are CPU-friendly but cause GPU memory divergence. This choice was tolerable when CPU and GPU memories were physically separate—you picked one format, and the cost of switching was dominated by PCIe transfer anyway. Unified memory removes the physical separation, and with it, the luxury of choosing one model over the other. A data structure on unified memory must serve both masters simultaneously: flat enough for GPU warp coalescing, mutable enough for CPU incremental updates. This is not merely an optimization problem—it is a structural inversion of the constraints under which virtually all classic graph data structures were designed. EdgeBlock is a first attempt at a format that balances these opposing demands. While this paper focuses on graph databases, the same challenge extends to

foundational data structures—B-trees, hash tables, sorting networks, and join algorithms will all need to be reconsidered under the dual constraints of unified memory.

We propose EdgeBlock, a block-based graph representation that addresses these challenges in a unified way:

1. Fixed-size blocks (32 edges) directly map to GPU warp width, enabling coalesced memory access without any format conversion.
2. CPU-GPU collaborative incremental algorithms that leverage EdgeBlock's dual-access nature: CPU manages the affected vertex queue and convergence logic, while GPU executes parallel computation in Metal compute kernels.
3. Zero-copy operation: Both CPU and GPU operate on the same `Vec<u32>` arrays in Rust, backed by Metal buffers with `StorageModeShared`.

Our proof-of-concept implementation, Axolotl-RS, currently validated on Apple M4 hardware (Metal GPU), is a Rust library with 77 unit tests, a REST API server, Python bindings (PyO3), and crash recovery via WAL. This paper describes the EdgeBlock design, the incremental algorithm framework, and presents performance measurements on Apple M4 hardware.

2. Background: Unified Memory Architecture

2.1 Apple M-Series Memory Model

Apple M-series processors (M1–M4) feature a Unified Memory Architecture (UMA) where CPU and GPU cores access the same physical DRAM pool. Unlike discrete GPUs (NVIDIA, AMD) where data must be explicitly transferred via `cudaMemcpy` over PCIe, UMA enables both processors to share pointers directly.

In Metal, the graphics API for Apple platforms, buffers are allocated with `MTLResourceOptions::StorageModeShared`, which places data in system memory

accessible to both CPU and GPU. This eliminates the need for explicit data marshaling but creates new design constraints:

- GPU memory access patterns still matter: non-coalesced access wastes bandwidth.
- CPU-side mutation of shared data requires synchronization (`wait_until_completed`).
- GPU compute kernels operate on threadgroups of 32 threads (the warp width).

2.2 Why CSR Falls Short

CSR (Compressed Sparse Row) is the most widely used graph format:

```
offsets: [0, 2, 4, 6, 8] // cumulative edge counts per vertex
targets: [1, 3, 0, 2, 1, 0, ...] // concatenated neighbor lists
```

For GPU PageRank, each thread processes one vertex and reads its in-neighbors from `targets[offsets[v]..offsets[v+1]]`. Because different vertices have different degrees, neighboring threads access non-contiguous memory regions, causing cache line thrashing.

More critically, in heterogeneous CPU-GPU processing, building CSR typically requires two passes: first to count degrees, then to populate offsets and targets. This intermediate step becomes the bottleneck—our measurements show CSR construction costs 61–83% of total processing time for graphs with 100K+ vertices.

2.3 Design Principles

Based on these observations, we established three design principles for EdgeBlock:

1. Single-format, zero-conversion: The same data structure serves both CPU traversal and GPU kernel execution.
2. GPU-friendly layout: Memory access patterns must respect warp coalescing (32 consecutive addresses).
3. Incremental-friendly: Updates (edge insertions) must be $O(1)$ amortized, and affected vertex detection must be efficient.

3. EdgeBlock Design

3.1 Block Structure

EdgeBlock organizes adjacency lists into fixed-size blocks:

Block Layout ($34 \times u32 = 136$ bytes):

ownerVertex (u32)	edgeCount	edge[0]	edge[1]	...	edge[31]
	(u32)	(u32)	(u32)		(u32)

```
BLOCK_CAPACITY = 32 // edges per block = GPU warp width
BLOCK_SIZE_U32 = 34 // 2 header + 32 edge slots
```

The complete graph is stored as:

```
GPUEdgeBlockGraph {
  vertices:    Vec<u32>, // v → first block index
  block_counts: Vec<u32>, // v → number of blocks
  blocks:      Vec<u32>, // flat array of all blocks (forward edges)

  reverse_vertices:    Vec<u32>, // same for reverse edges
  reverse_block_counts: Vec<u32>,
  reverse_blocks:      Vec<u32>,

  idx_to_id: Vec<u64>, // internal index → external ID
  id_to_idx: HashMap<u64, usize>, // external ID → internal index

  vertex_props: Vec<HashMap<String, PropertyValue>>,
  edge_data:    HashMap<(u64, u64), EdgeData>, // edge properties

  dirty_flags: { blocks_dirty, reverse_blocks_dirty, structure_dirty }
}
```

Both forward and reverse adjacency are maintained. The forward edge blocks support out-neighbor traversal (BFS walk, subgraph extraction). The reverse edge blocks support in-neighbor traversal, which is critical for PageRank computation.

3.2 GPU Memory Access Pattern

When GPU launches a PageRank kernel, each thread processes one vertex `v`:

```
Thread gid=0 → vertex 0 → reads reverse_blocks[reverse_vertices[0] .. ]
Thread gid=1 → vertex 1 → reads reverse_blocks[reverse_vertices[1] .. ]
...
```

Because adjacent threads process adjacent vertices (gid 0, 1, 2, ...), and EdgeBlock stores all vertex blocks contiguously, the GPU's memory controller can coalesce reads when consecutive vertices happen to have their blocks in the same cache-line-sized region. More importantly, within a single block, all 32 edge targets are contiguous, so a single cache line fetch (128 bytes) loads the entire block.

3.3 Edge Insertion: $O(1)$ Amortized

```
fn add_edge(from, to):
    last_block = vertices[from] + block_counts[from] - 1
    count = blocks[last_block + 1]    // edgeCount header

    if count < 32:
        blocks[last_block + 2 + count] = to
        blocks[last_block + 1] = count + 1    // 0(1) most of the time
    else:
        blocks.push(ownerVertex=from, edgeCount=1, to, 0, ..., 0)
        block_counts[from] += 1    // 0(1) amortized
```

For 95% of insertions (block not yet full), the operation is a single array write. When a block reaches 32 edges, a new block is appended—amortized $O(1)$. The `structure_dirty` flag signals the GPU that `vertices` / `block_counts` arrays have changed and Metal buffers need re-encoding.

3.4 Vertex Deletion: Lazy Marking

Rather than physically removing blocks, we mark the vertex ID as invalid (`idx_to_id[i] = u64::MAX`). Edge blocks belonging to deleted vertices are skipped during traversal and GPU computation. This avoids the $O(n)$ cost of rebuilding the block array.

3.5 Binary Persistence: AXEB Format

AXEB File Layout:

AXEB	version	vertex_cnt	total_edges	← Header (16 bytes)
Section 1: Topology				
vertices, block_counts, blocks (fwd)				
reverse_vertices, reverse_block_counts,				
reverse_blocks				
Section 2: ID Mapping				
idx_to_id[]				
Section 3: Vertex Properties				
count, key/value pairs per vertex				
Section 4: Edge Properties				
count, (from, to, weight, key/value)				

The file is written atomically (temp file → rename) to prevent corruption. A Write-Ahead Log (WAL) records all mutations for crash recovery.

4. Incremental Algorithm Framework

4.1 CPU–GPU Collaboration Pattern

Our framework follows a unified loop across all five algorithms:

```
Algorithm: IncrementalUpdate(graph, changed_edges)
    affected = []                                // CPU
    for (u, v) in changed_edges:
        graph.add_edge(u, v)                    // CPU: O(1)
        affected.push(u); affected.push(v)

    while not converged:
        new_affected = []
        gpu_results = launch_kernel(graph, affected) // GPU: parallel
        for v in affected:
            if result_changed(v, gpu_results):
                new_affected.push(neighbors_of(v)) // CPU: sequential
```

```

affected = deduplicate(new_affected)           // CPU

if |affected| < threshold or iteration > max_iter:
    converged = true

```

CPU responsibilities: – Maintain the affected vertex set (`HashSet<VertexId>`) – Check convergence (compare old vs. new values) – Propagate changes: if vertex `v` changes, its neighbors become affected – Union–Find operations (Connected Components)

GPU responsibilities (Metal compute kernel): – Process each affected vertex in parallel (one thread per vertex) – For PageRank: compute weighted sum of in–neighbor PR values – For BFS: compute minimum distance from affected neighbors – For SSSP: edge relaxation from affected neighbors – For Connected Components: batch find–root operations

4.2 PageRank

The PageRank kernel handles dangling vertices (out–degree = 0) correctly—a detail often overlooked in naive implementations that causes PR sum to diverge from 1.0:

```

kernel void pagerank_edgeblock_optimized(
    device const uint  *affected_vertices,
    constant uint      &affected_count,
    device const uint  *reverse_vertices,
    device const uint  *reverse_block_counts,
    device const EdgeBlock *reverse_blocks,
    device const float *pr,
    device float       *new_pr,
    device const uint  *out_degrees,
    constant float     &damping_factor,
    constant uint      &vertex_count,
    constant float     &dangling_contribution,
    uint gid [[thread_position_in_grid]]
) {
    if (gid >= affected_count) return;
    uint v = affected_vertices[gid];

    float contribution = 0.0;
    uint block_start = reverse_vertices[v];
    uint block_end = block_start + reverse_block_counts[v];

    for (uint bi = block_start; bi < block_end; bi++) {

```



```

    EdgeBlock block = reverse_blocks[bi];
    for (uint i = 0; i < block.edge_count; i++) {
        uint src = block.edges[i];
        uint od = out_degrees[src];
        if (od > 0) contribution += pr[src] / float(od);
    }
}

float base = (1.0 - damping_factor) / float(vertex_count);
new_pr[v] = base + damping_factor * (contribution + dangling_contribution);
}

```

4.3 BFS

The BFS kernel is simpler—it only needs to check if any in-neighbor’s distance has changed:

```

kernel void bfs_edgeblock(
    device const uint  *affected_vertices,
    constant uint      &affected_count,
    device const uint  *reverse_vertices,
    device const uint  *reverse_block_counts,
    device const EdgeBlock *reverse_blocks,
    device const float *distances,
    device float       *new_distances,
    uint gid [[thread_position_in_grid]]
) {
    uint v = affected_vertices[gid];
    float min_dist = INFINITY;

    // Check all in-neighbors for a shorter path
    for each block in reverse_blocks[v]:
        for each neighbor src:
            if distances[src] + 1 < min_dist:
                min_dist = distances[src] + 1;

    new_distances[v] = min_dist;
}

```

The CPU then checks if `new_distances[v] < distances[v]`; if so, all out-neighbors of `v` are added to the affected set for the next iteration.

4.4 Connected Components

Connected Components uses a Union–Find data structure maintained by the CPU. The GPU kernel batch–processes find–root operations:

```
kernel void cc_find_roots(
    device const uint *affected,
    device const uint *parents,
    device uint      *roots,
    uint gid [[thread_position_in_grid]]
) {
    uint v = affected[gid];
    // Path compression on GPU
    while (parents[v] != v) {
        v = parents[v];
    }
    roots[gid] = v;
}
```

After GPU returns root IDs for affected vertices, the CPU performs Union operations and detects newly affected vertices.

5. Performance Evaluation

5.1 Experimental Setup

Parameter	Value
Hardware	Apple M4, 10 GPU cores
Memory	Unified Memory, <code>StorageModeShared</code>
OS	macOS 14+
Language	Rust 1.75+
GPU API	Metal 3.2 (via <code>metal-rs</code>)
Threadgroup size	32 (matching warp width)

Test graphs (power–law degree distribution):

Dataset	Vertices	Edges	File Size
Small	1,000	5,000	72 KB
Medium	10,000	50,000	895 KB
Large	50,000	250,000	—

5.2 Data Loading Performance

EdgeBlock eliminates the CSR construction bottleneck:

Dataset	CSR Pipeline	EdgeBlock	Savings
1K/10K	4.74ms	0.79ms	83%
10K/100K	40.93ms	8.14ms	80%
100K/1M	745.53ms	293.20ms	61%

5.3 Incremental Algorithm Speedup

We measure speedup as $\text{full_recomputation_time} / \text{incremental_update_time}$ when adding 50 random edges:

Algorithm	1K Vertices	10K Vertices	50K Vertices
BFS	31x	283x	1580x
Connected Components	88x	720x	4920x
PageRank	0.1x	7.5x	18.9x

BFS and CC show extreme speedups because the change propagation is highly localized—adding 50 edges affects only a small neighborhood. PageRank shows more modest speedup because even a single edge change creates a global effect that requires propagation through all vertices, making incremental updates comparable to full recomputation for dense graphs.

Note on speedup ratios: The extreme speedups for BFS (1580x) and CC (4920x) reflect the best-case scenario where only 50 out of 50,000 vertices are directly affected by edge additions. In real-world workloads where a larger fraction of the graph changes, the speedup ratio decreases. These numbers establish an upper

bound, not an average-case expectation. All measurements are Rust-vs-Rust (not cross-language comparisons), with both full and incremental algorithms running on the same EdgeBlock graph structure. EdgeBlock’s block headers introduce a small overhead (~20%) for full-graph traversals vs. CSR, meaning the full recomputation baseline is slightly slower than an optimal CSR implementation. A CSR-based baseline would yield ~1317x for BFS and ~4100x for CC—still the same order of magnitude. Crucially, CSR cannot support incremental updates without format conversion, so a CSR-vs-CSR incremental comparison is not feasible.

5.4 GPU Buffer Cache Reuse

By caching Metal buffers across kernel calls, we avoid re-allocation overhead:

Algorithm	1K/5K	10K/50K	100K/1M
PageRank	22%	25%	25%
BFS	21%	24%	24%
SSSP	24%	24%	25%

5.5 PageRank Correctness

All implementations produce PR sum = 1.0000 with maximum per-vertex error < 10⁻⁶, verified across three implementations (CPU correct, GPU incremental, GPU full recomputation).

6. Related Work

GPU graph frameworks. Gunrock [1] provides a high-level API for GPU graph algorithms using CSR format, targeting CUDA architectures. cuGraph (NVIDIA RAPIDS) offers GPU-accelerated graph analytics in Python, also CSR-based. Both require explicit `cudaMemcpy` for data transfer.

Incremental graph algorithms. GraphBolt [2] supports incremental updates by tracking affected vertices, similar to our approach, but targets discrete GPUs. KickStarter [3] trims dependency graphs for incremental computation. Our

contribution is the convergence of both ideas—incremental algorithm framework + unified memory format—into a single design.

Unified memory graph processing. Ligra [4] is a shared-memory graph processing framework but targets multi-core CPUs, not GPU. Totem [5] explores GPU graph processing on unified memory but uses standard CSR.

Block-based graph formats. The idea of fixed-size edge blocks has been explored in Cagra [6] for GPU-based graph indexing. EdgeBlock differs in being bidirectional (both forward and reverse adjacency) and designed for incremental updates rather than static indexing.

To our knowledge, EdgeBlock is the first graph format that: 1. Is designed from the ground up for unified memory (not ported from CUDA), with initial validation on Apple Metal. 2. Supports both forward and reverse adjacency in the same array structure. 3. Enables incremental algorithms without any intermediate format conversion.

7. Implementation

The full implementation, Axolotl-RS v0.1.0-beta, is available at github.com/LittleLollipop/axolotl under the MIT license. It includes:

- 77 unit tests covering storage, CRUD, persistence, algorithms, and recovery.
- REST API server with 16 endpoints, 16-thread pool, non-blocking accept.
- Python bindings via PyO3 + maturin (pip-installable).
- Crash recovery via WAL (Write-Ahead Log) with automatic replay.
- AXEB binary format for persistence and portability.

```
prototype-rust/  
├─ src/  
│   ├─ gpu_edge_block.rs    # EdgeBlock core (1894 lines)  
│   ├─ graph_db.rs          # Unified interface  
│   ├─ server.rs            # REST API  
│   ├─ py_bindings.rs       # Python API  
│   ├─ recovery.rs          # WAL crash recovery  
│   ├─ transaction.rs       # Transaction support  
│   └─ mvcc.rs              # MVCC snapshot isolation
```

```
| |─ mmap_graph.rs      # Memory-mapped graphs
| |─ pagerank_correct.rs # Correct PageRank
| |─ incremental_*.rs    # 5 incremental algorithms
| |─ gpu/mod.rs          # GPU accelerator
|─ examples/
| |─ server.rs           # REST API server
| |─ bench_incremental.rs # Algorithm benchmarks
|─ benches/              # Criterion benchmarks
```

8. Conclusion and Future Work

We presented EdgeBlock, a fixed-size block graph format designed for unified memory architectures, and an incremental algorithm framework achieving up to 4920× speedup (Connected Components) and 1580× speedup (BFS) over full recomputation. The core insight—aligning block size to GPU warp width to eliminate both format conversion overhead and non-coalesced memory access—is validated on Apple M4 hardware and applicable to any unified memory platform.

Future work includes:

- PageRank optimization: Improve incremental GPU kernel efficiency for denser graphs.
 - Larger-scale testing: Extend benchmarks to 10M+ vertex graphs.
 - Distributed EdgeBlock: Partition the block array across multiple machines while preserving the block format.
 - Formal analysis: Prove convergence bounds for the incremental propagation loop.
-

References

- [1] Y. Wang et al., “Gunrock: A High-Performance Graph Processing Library on the GPU,” PPoPP 2016.
- [2] M. Mariappan et al., “GraphBolt: Dependency-Driven Synchronous Processing of Streaming Graphs,” EuroSys 2021.

- [3] K. Vora et al., “KickStarter: Fast and Accurate Computations on Streaming Graphs via Trimmed Approximations,” ASPLOS 2017.
- [4] J. Shun and G. E. Blelloch, “Ligra: A Lightweight Graph Processing Framework for Shared Memory,” PPOPP 2013.
- [5] A. Gharaibeh et al., “Totem: A GPU-Enabled Hybrid Graph Processing Engine,” TPDS 2017.
- [6] H. Ootomo et al., “CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs,” ICDE 2024.
-

This is a pre-submission draft. Feedback welcome at the project repository.